

GPU HA Whitepaper

GPU HA: Two-Tier Failover for LLM Inference — What DNS Can and Can’t Do for You

Scott McDonald · Draft v0.3 (2026-07-03) · Status: PRIVATE — do not publish before the patent/open-source decision (§8 mechanism detail is the sensitive element; see OPEN_CORE.md for the disposition table)

*Everything below describes built, tested, running code unless explicitly marked. The two-tier telemetry contract is proven across **three independent transport legs — localhost, LAN, and public internet**, and intra-pool failover is proven against **real GPUs** at both process and host isolation levels. The multi-cloud orchestrator is built and drilled. Remaining deferred items are production-hardening and breadth, marked **[D2]** (production DNS wiring) and noted inline; they are not correctness gaps.*

1. Why I built this

Twenty-some years ago I built dnshat.com — DNS-based failover for websites — at a time when people said DNS caching made it impossible. It worked, it was popular because it worked, and it led to an AWS acquihire. When GPU inference became the workload everyone’s availability story depends on, the obvious question was whether the same trick works again: can DNS route AI traffic around dying GPUs?

The honest answer, which this project exists to demonstrate rather than assert, is: **half of it can**. The half that can’t is the interesting half, and the boundary between them is precise enough to draw in code. GPU HA is the working proof — a two-tier failover system for LLM inference, built and drilled on real GPUs across three clouds (GCP, RunPod, Lambda) and running today on a permanent always-on control plane, as a learning vehicle for the modern inference stack (vLLM, multi-cloud GPU capacity, L7 streaming proxies) and a portfolio piece.

2. The thesis: where DNS’s reach ends

Website failover suited DNS because the failure it routed around was **coarse and sticky** — an origin died, and new connections needed to land elsewhere within seconds-to-a-minute. Resolver caching hurt a little; it didn’t matter much, because requests were short and stateless. A stale answer cost one failed page load and a retry.

LLM inference breaks that model in two specific ways:

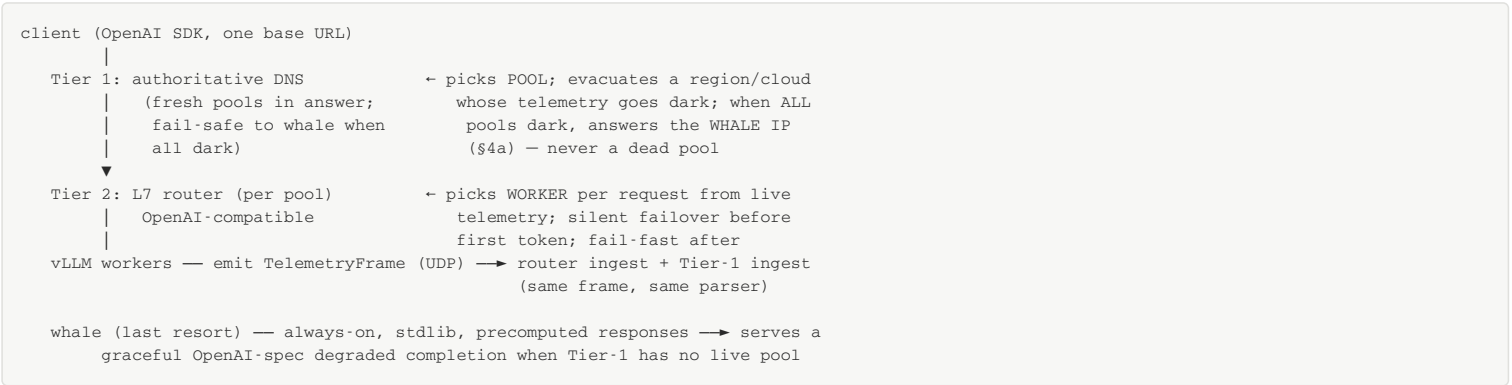
The connection outlives the decision. A streaming completion holds a connection open for 5-90 seconds. DNS resolves once, at connection setup; the moment the socket opens, DNS is out of the loop. No TTL is low enough to re-route a request that is already in flight.

The decision is fast-moving and per-request. “Which node has the lowest time-to-first-token and the most VRAM headroom *right now*” changes second to second. Public resolvers override TTL 0 with 20-60s minimum caching, freezing a per-second decision for a minute — for every client behind that resolver.

So the design splits cleanly: **DNS (Tier 1) owns what it is unbeatable at** — coarse pool evacuation: a whole region or cloud losing capacity, telemetry going dark, cross-cloud disaster failover. **An L7 hop (Tier 2) owns the per-request decision** — which specific worker serves this request, and what happens when that worker dies mid-stream. The dnshat lesson wasn’t “DNS solves everything”; it was “DNS solves that specific failure mode well.” The discipline is knowing which one you have.

3. Architecture

One versioned telemetry frame drives both tiers — that single contract is the load-bearing design decision (§8).



Tier 2 components, all stdlib Python (a deliberate constraint — the code runs anywhere, and the failover logic stays legible rather than buried in framework config):

- **Selector** — health gate before scoring (a worker with stale telemetry is *dead*, never “idle and available”), transparent weighted scoring (TTFT and VRAM headroom dominate; price is an intra-pool tiebreaker), a per-worker circuit breaker, and anti-herding (squared-score sampling among the top N, so the best worker is preferred without being stampeded).
- **Router** — one OpenAI-compatible endpoint; walks the selector’s retry plan; enforces the first-token boundary (§4); handles both chunked and Content-Length upstream framings.
- **Telemetry shim** — scrapes a *stock* vLLM server’s Prometheus `/metrics`, packs the numbers into the versioned frame, pushes UDP. The GPU box stays unmodified vendor software; all custom code lives on a cheap CPU box.

Tier 1 (`tier1_dns.py`, built and drilled in Phase D) is a minimal authoritative DNS responder that imports the **same ingest class** the router uses — shared code, not merely shared format. A pool appears in the answer iff at least one of its nodes has fresh telemetry within a deliberately coarser window (10s pool eviction vs. the router’s 3s node gate — evacuation should be slower-twitch than per-request selection); if every pool is dark, it fails **open** to a configured static-safe set — never an empty answer, never NXDOMAIN, never stale state served as live. That fail-open choice closes a question left open in the original design: when telemetry dies, a DNS tier that keeps confidently serving its last opinion is lying, and one that returns nothing takes the service down itself; serving the full known set and logging the degradation is the only honest option.

A lineage note: this project’s ancestor — a custom CoreDNS hidden master with a native Go plugin (`gpuha.go`), built in an earlier R&D iteration — was recovered during Phase L. Its telemetry model is the direct ancestor of the current design and its instructive *contrast*: a central aggregator daemon hardcoded fleet state and UDP-broadcast it to the master, which scored nodes and rewrote a zone file, firing RFC-1996 NOTIFY to five anycast slaves. Two things the ancestor makes concrete by contrast: (1) it did per-request “best node” selection *in DNS* — rewriting an A record to one winning IP per telemetry cycle — the exact responsibility the two-tier design deliberately moved to the L7 router; and (2) its liveness was a central aggregator’s `last_seen` stamp, a single point of failure, versus the POC’s consumer-judged arrival freshness off a versioned, content-free frame with no central aggregator at all. A small, almost poetic detail surfaced in recovery: the ancestor’s live zone file still resolved `api.gpuha.com` to a Lambda IP that no longer existed — the old engine’s *last computed routing decision*, frozen in DNS, outliving the infrastructure it pointed at. DNS state is sticky and survives what it describes, which is, after all, half this project’s thesis. The recovered source is preserved (privately) as prior-work reference; wiring a native TelemetryFrame parser into CoreDNS is deferred production polish (D2). The POC’s Tier-1 deliberately re-proves the mechanism in ~200 lines of the same stdlib Python as the rest of the stack, importing the *same ingest class the router uses*.

4. The first-token boundary — the honest failover contract

The central claim of any streaming-inference HA system has to answer: *what exactly happens when a worker dies mid-generation?* Most marketing hand-waves this. The truthful contract is:

- **Failure before the first token reaches the client:** fully recoverable. The router silently retries the next-best worker; the client sees nothing but slightly higher TTFT.
- **Failure after the first token:** *not* transparently recoverable. Tokens already streamed came from one worker’s KV-cache state; no other worker shares it. Pretending to resume would mean silently regenerating different output. The honest behavior is a clean truncation and a client-level retry of the whole request.

The router enforces this as control flow — a `committed` latch flips the instant the first content byte is forwarded, and no code path retries after it. In the Phase A drills (§6) both sides of the contract were exercised against a real GPU being killed.

(Production note: shared/disaggregated KV-cache designs could move this boundary. That’s a research-grade extension, out of POC scope, and worth naming precisely *because* the current contract is honest about it.)

4a. The fail-whale — graceful degradation when everything is dark

The first-token contract governs *a* pool with *some* live worker. But what does a client get when **every** pool is dark — all GPUs gone, all telemetry silent? The naive answer is what Tier-1’s fail-open originally did: keep the pools in the DNS answer, and let clients hit dead router IPs. Honest at the DNS layer, broken at the client — connection-refused and SDK stack traces.

The fix borrows a pattern from the dnshat era. When Twitter couldn’t serve traffic, DNS failed over to a static “fail whale” page — the user saw something human, not a browser error. Rebuilt for OpenAI-spec token traffic: a last-resort endpoint (`whale.py`) that speaks the protocol and answers when nothing else can. Tier-1’s FAILSAFE now points at the whale IP instead of at dead pools. The whale serves either a **protocol-correct 503 + Retry-After** (which official SDKs auto-retry, the self-healing path for API clients) or, for chat apps, a **valid 200 streamed completion** whose content is a polite “give me a moment, I’m having resource trouble” message — signaled in-band via `"model": "gpuha-degraded"` so naive frontends render the text while careful clients detect degradation from a field they already parse.

Governing design principle: **the whale is the dumbest thing in the fleet**. No telemetry, no state, no per-request work, no dependencies — every response is a byte buffer precomputed at startup. A last resort that shares fate with the things it’s a last resort *for* is decoration. Measured ~5k req/s single-process (stdlib only); scales out via SO_REUSEPORT.

This was proven end-to-end across the public internet in Phase D.5 (§6): with both pools killed, a real external client followed DNS to the whale IP and received HTTP 200, `"model": "gpuha-degraded"`, and the graceful message — never a connection error. The complete failure arc — pools dark → DNS evacuates → DNS fails to whale → client gets a courteous OpenAI-spec answer — runs on real infrastructure.

The honest residual (same one dnshat had): clients holding a cached DNS answer (public-resolver TTL floors of 20–60s) still hit dead pool IPs until their cache expires. The whale covers the post-TTL experience and the router-local “my pool is dead but I’m alive” case; a fully dead pool host answers nothing until TTL expiry. Anycasting the whale IP is the deferred production mitigation. The whale does not claim to close the cached-DNS window — it closes everything downstream of it.

5. Liveness is the consumer’s judgment: silence = death

The single most important HA decision in the system is negative-space: **no worker ever asserts “I am healthy.”** Frames carry raw metrics plus identity and sequence; each consumer judges liveness by *its own clock* against frame arrival. A node that stops emitting ages out of the eligible set within seconds — no error required, no health-check endpoint to lie.

This paid for itself immediately in drills: a killed vLLM leaves an orphaned subprocess holding all GPU memory (\$7.3). A “is the process up / is the port open” health check misreads that state; arrival-freshness cannot be fooled by it, because the shim’s scrape fails and emission simply stops.

Two supporting invariants that turned out to matter: the emitter’s scrape timeout must sit well below the consumer’s freshness window (a scrape that can block for 3s against a 3s window makes a merely *slow* worker flap in and out of eligibility — slow ≠ dead; slowness is what the TTFT score is for), and breaker-cleared and telemetry-fresh are independent gates — a recovered worker needs both.

6. Drill results

Phase A — Tier-2 failover against a real GPU (RunPod, L4, vLLM 0.11)

Drill	Result
Pre-token failover: <code>kill -9</code> vLLM during live traffic	5/5 requests completed with 200s, transparently served by fallback workers; no client-visible error
Silence pruning: telemetry stops	worker dropped from eligible set within the freshness window, no request had to fail first
Mid-stream fail-fast: kill during generation, post-commit	client stream cleanly truncated (no fake resume, no counterfeit [DONE]); fail-fast recorded
Recovery: restart vLLM	worker re-entered eligible only after breaker cooldown AND fresh telemetry — both gates

Honesty note: the *dying* worker was a real L4; the failover *targets* were protocol-identical stubs, pending multi-GPU quota. That asterisk is removed below (\$6, “Real-GPU intra-pool failover”) — the failover targets are now real GPUs.

Real-GPU intra-pool failover — closing the stub asterisk (Phase C’)

Phase A proved a real GPU *dying* but with protocol-identical stubs as the failover *targets*. Two later drills replaced those stubs with real GPUs, at two levels of isolation:

- **Process-level (co-resident).** Two real vLLM workers (Qwen2.5-3B) co-resident on a single Lambda A10 behind one router. Killing one worker’s vLLM mid-traffic → the router evicted it on telemetry silence (`eligible: [w1, w2] → [w2]`) → every subsequent completion was served by the survivor, real tokens, zero client errors, the `X-GPUHA-Served-By` header flipping accordingly. This proves the failover target is a real GPU worker serving real tokens — but both workers shared one physical card, so it is *process-level* failure, not host isolation.
- **Host-level (separate instances).** The stronger version: two workers on two *separate* Lambda A10 instances, a router-head on instance A fronting both A’s local worker and B’s worker over the same-region private network (B’s vLLM port firewall-scoped to A’s address only — the router-head is the sole legitimate client of it, verified with a negative test: the port is unreachable from any other host). Terminating B’s *entire instance* mid-traffic → the router evicted B on silence and served all post-kill requests from A’s worker on the other machine. That is true host-level failover; it closes the co-residency residual.

Cold-load baseline captured here for reference: on an A10, `vLLM serve → serving-ready` was ~63s for Qwen2.5-3B (weights ~11s, engine/KV/CUDA-graph warmup ~36s); the longer pole was cloud instance provisioning, not the software path.

One honest failure the host-level drill exposed — and the fix. In the first host-level run, exactly one request out of 200 failed at the kill instant: it rode the 8-second client timeout instead of failing over. Root cause is precise and instructive: the router’s retry path fired on *connection errors* (refused/reset), but a terminating host briefly *accepts-then-hangs* — so that one in-flight request hit a socket that accepted and then went silent, which is neither a connection error nor a delivered first token. It fell into the gap between the two failure modes the retry logic knew about. The fix: a per-attempt upstream deadline (`PRE_TOKEN_DEADLINE`) wraps the connect and the first pre-token body chunk; a stall now *is* a pre-token failure → trip the breaker, retry the next-best worker, increment the failover counter. Re-running the exact drill with the patched router: **0 client errors out of 260**, the kill-instant request visibly stalling ~4s to the deadline, retrying, and returning HTTP 200 — the failover now counted where before it was invisible. The first-token contract now covers *both* “connection refused” and “silent-but-not-refused.” (This is the same lesson as §7 in miniature — see §7.8.)

A second cloud, and a NAT’d worker. The host-level pattern above was then reproduced on a second, independent cloud (RunPod) — and RunPod adds a real wrinkle: its workers sit behind a NAT’d TCP proxy, so a remote worker cannot push UDP telemetry to a sibling pool’s router the way two same-region hosts can. Rather than weaken the liveness contract with an active health-poll, the fix keeps the contract intact by *relocating who emits the frame*: the router-head host runs the remote worker’s telemetry shim itself, scraping the remote worker across the proxy and emitting normal frames to its local router — so “silence = death” fires unchanged when the remote worker dies, because the scrape simply stops. The exposed proxy port (public by design on RunPod, where source-IP firewalling is impossible) is protected instead by an API key: the worker’s inference server requires it, the router injects it, and an unauthenticated request to the public port is rejected — verified as a hard gate (no-key → 401, with-key → 200, read simultaneously from outside the pool). With that in place, a real pod-delete of the remote worker mid-traffic gave **88/88 client requests served with 200, zero errors**, the remote worker evacuated on silence, and traffic flipped to the surviving worker — host-level intra-pool failover across a NAT’d

worker on a second cloud, with the liveness contract and the “no unauthenticated GPU” invariant both preserved. Finally, a combined two-cloud topology proved **cross-cloud evacuation** at the Tier-1 layer: killing one cloud’s entire pool dropped it from the DNS answer within ~15s, leaving the other cloud serving, and with both pools dark the answer became the whale (\$7.9 states precisely which leg of this was proven at the completion layer versus the DNS/telemetry layer).

Phase D / D.5 — Tier-1 evacuation, proven across three transport legs

The Tier-1 mechanism was proven in three stages, each adding a harder transport reality: **localhost** (single host, two simulated pools), **LAN**, and finally **public internet** — a Seattle node firing TelemetryFrames to a Miami plane. The cross-internet drill (D.5) is the one that matters, because it is the first time frames crossed NAT, real peering, and firewall paths rather than a loopback:

State	<code>dig api.gpuha.com</code> returns	tier1 log	dropped_frames
Baseline (both pools fresh)	plane IP + remote-pool IP	<code>ANSWER SET -> [plane, linode-west]</code>	<code>{}</code>
Evacuate (remote emitter killed)	plane IP only	<code>ANSWER SET -> [plane]</code>	<code>{}</code>
Restore (remote emitter back)	both again	<code>ANSWER SET -> [plane, linode-west]</code>	<code>{}</code>
FAILSAFE (both pools dark)	whale IP alone	<code>[FAILSAFE->WHALE <plane-ip>]</code>	<code>{'linode-west': 185}</code>

Two results worth dwelling on. First, **zero UDP loss on the live cross-internet path** (`dropped_frames={}` through baseline/evacuate/restore) — a clean transport result over real public internet, not loopback. Second, and more telling: the seq-gap count of **185 appears only at FAILSAFE**, i.e. only after the remote node went silent. That is the freshness gate proving itself with a number attached — silence registered as accumulated sequence gaps, the node aged out, and the system degraded to the whale instead of continuing to advertise a dead pool. “Silence = death” is not a slogan here; it is an observable count.

Final proof, from the remote node acting as an external client that followed DNS to the whale: `curl` returned HTTP 200, `"model": "gpuha-degraded"`, and the graceful message. A real cross-internet client, with every backend dead, received a courteous OpenAI-spec answer rather than a connection failure. That closes the distributed proof.

Honesty note: these drills used lightweight stub emitters as the *pools* (the point of D.5 was the DNS/transport/whale layer, which is content-agnostic — it evacuates on telemetry silence and never inspects inference). Real-GPU intra-pool failover — the evacuation target being a real GPU rather than a stub — is proven separately (see “Real-GPU intra-pool failover” above). What D.5 proves specifically: the full Tier-1 evacuation + whale mechanism, over real public-internet transport, with real DNS queries and the shared ingest class. The two proofs are complementary — D.5 is the cross-cloud DNS/whale layer; the C’ drills are the real-GPU worker-to-worker layer.

7. What reality taught us that the tests didn’t

The unit and integration suites (five simulated scenarios, five telemetry-contract tests) caught real design bugs before any cloud spend — notably that linear score-weighting washes out small quality gaps into coin-flips (fixed by squaring), and the breaker/freshness gate independence. But four bugs only surfaced against real infrastructure, and they’re the most instructive part of the project:

7.1 The sequence lockout. Frames carry a monotonic per-node `seq`; the ingest rejects `seq <= last_seq` to defend against UDP reordering. A *restarted* emitter resets seq to 0 — and is therefore locked out forever, indistinguishable from replayed packets. Reality: every emitter eventually restarts. POC fix: seed seq from wall-clock. Principled fix (frame v2): an epoch/boot-id field so the ingest can distinguish “reordered datagram” from “restarted emitter.”

7.2 Framing dispatch. The router originally assumed chunked transfer-encoding, because the test doubles always streamed. Real vLLM only chunks when the request asks for `stream: true`; buffered responses arrive with Content-Length, which the chunk parser read as garbage — producing *false breaker trips* against a perfectly healthy worker. An HA layer whose own parsing bug marks healthy workers dead is a self-inflicted outage. Lesson: test doubles inherit your assumptions; the real dependency is the test.

7.3 VRAM zombies. `kill -9` on vLLM orphans its EngineCore subprocess, which keeps holding ~all GPU memory — the *node* looks restartable while the *GPU* is occupied, and the restart fails on memory. This is a genuine GPU-HA failure class: process liveness and resource liveness diverge. Silence-based liveness handled it correctly (\$5); a telemetry-level zombie signature (VRAM ~full with no serving process) is logged as a frame-v2 detection note.

7.5 The infrastructure fought us harder than the code did. A recurring, humbling pattern: more time was lost to *transport and tooling* than to the actual distributed-systems problem. A browser-driving agent kept reaching for browser-shaped doors — driving a serial console by screenshot, pasting files as base64 chunks through a terminal — when a real shell or an upload button was right there. Cloud Shell silently blocked outbound UDP (invisible until it broke a transport test) and disconnected on long operations. A stopped Linode still bills (unlike AWS/GCP), quietly draining ~\$68/mo across five forgotten boxes. A custom-named SSH deploy key isn’t offered by default, producing “the key is definitely on the repo but still permission denied” until an `~/.ssh/config` IdentityFile entry fixes it. None of these are glamorous, and all of them cost real hours. The lesson worth generalizing: for a project whose entire premise is operational resilience, the operational friction is the curriculum — the scars became a per-provider lifecycle matrix, and that matrix becomes product code.

7.6 An agentic build workflow, honestly assessed. This was built with a three-role split: a human supervisor (decisions, approvals, credentials), a reasoning model as architect (design, briefs, code artifacts, review), and a browser-capable agent as engineer (drives consoles/UIs, runs commands). Briefs are the interface; anything irreversible (deletions, spend) is human-gated; credentials never pass through an agent (keys generated by the human, public halves only; root passwords typed by the human). What worked: the split kept the architect out of infra and the engineer out of redesign, and the agent’s best moment was refusing to claim a capability it didn’t have

— told to “just SSH in,” it *tested*, found its sandbox had no egress and never held the private key, and said so, instead of pretending. What didn’t: browser agents default to browser-shaped solutions even when worse; the “bootstrapping gap” on a fresh box (no shell an agent can reach until you stand one up) is inherently the human’s to cross; and version control is the only real cure for the divergent-copies problem that otherwise metastasizes across sandbox, connected folder, and box. Verify-don’t-claim is the rule that most improved outcomes — for the models and the human alike.

7.7 The crash creates resources your own state file never knew about. Building the orchestrator’s teardown safety, we ran a deliberate kill test: `SIGKILL` the orchestrator mid-`create`, while it was concurrently bringing up two pools. The persisted run-state came back *empty* (`resources=[]`) — correct by its own logic, because the adapters create the cloud resource *before* persisting the handle. But the detached provider-side create calls finished anyway and left two live instances as orphans, invisible to the state file. State-based teardown would have missed them entirely and billed until noticed. The independent, label-based `reap --all` — which queries each provider for anything tagged `gpuha-managed` rather than trusting local state — found and destroyed both, returning the fleet to \$0. The lesson generalizes the whole safety philosophy: **provider truth, not state truth, is the only safe basis for teardown.** A backstop that reads the same state the primary path reads is not independent; the reap earns the word “backstop” precisely because it consults an authority the crash couldn’t corrupt. (Principled fix — persist-before-create ordering — is filed as hardening; the label reap makes it non-urgent, because the safety net already covers the window the ordering bug opens.) This is the billing-safety twin of the liveness principle in §5: just as a worker’s *silence* is judged by an outside clock rather than its own health claim, a leaked resource is caught by the *provider’s* inventory rather than the orchestrator’s memory of what it thinks it created.

7.8 “Refused” and “silent-but-not-refused” are different failures. The first-token contract (§4) says failure before the first token is silently recoverable. The implementation honored that — for *connection* failures: a worker that refused or reset the connection got retried. But a real host being torn down doesn’t refuse; it briefly *accepts-then-hangs*. In a host-level failover drill (§6), exactly one request in 200 hit that state at the kill instant — a socket that accepted and then went silent — and it fell into the gap between the two failures the retry path recognized, riding the client timeout instead of failing over. The fix was a per-attempt upstream deadline: a stall before the first token is now *treated as* a pre-token failure and retried, and the re-run served 260/260 with zero client errors. The lesson is the same shape as the VRAM zombie (§7.3: process-liveness and resource-liveness diverge) and the orphan-reap (§7.7: state-truth and provider-truth diverge): **two things you assumed were one thing are actually two.** “The worker is unreachable” turned out to be at least two distinct events — refused and hung — and an honest contract has to cover both, not just the one the mocks happened to produce. By this point in the project the pattern was so well-established it had become predictive: the gap was found by asking “what failure mode would a mock never generate?” rather than by waiting for it to bite.

7.9 The provider moves the territory under your map. The final instance of the pattern did not come from the code at all — it came from a cloud provider silently changing a default. In the combined multi-cloud drill, a Lambda pool and a RunPod pool were brought up under one Tier-1. Both pools registered in DNS, both passed telemetry, the RunPod pool served completions — and the Lambda pool’s *completion* path was unreachable from the outside, because Lambda’s cloud now blocks inbound `:9000` by default. The instructive part is the exact shape of the divergence: the Lambda router was healthy *locally* (listening, vLLM returning 200), and its *outbound* telemetry reached the plane fine (which is why it correctly registered as a live pool in Tier-1), but the *inbound* completion port was firewalled. So the DNS tier’s map said “this pool is alive” — truthfully, by the only signal it consumes, telemetry arrival — while the territory said “you cannot actually reach it to serve a request.” This is the deepest version of the whole project’s recurring lesson: not “the mock differs from the real dependency,” but “**the real dependency’s behavior changed after you validated it**, because it is owned by someone else who changed a default.” The cross-cloud evacuation and whale behaviors were therefore proven at the Tier-1/DNS/telemetry layer (which is what actually performs evacuation — it runs on telemetry, and the telemetry path was unaffected) rather than with a live Lambda completion handoff. That boundary is stated precisely rather than papered over (§9). The fix is trivial and provider-specific — a Lambda inbound rule for `:9000` — and deliberately *not* pursued, because it would harden a throwaway demo rig’s provider-specific path rather than teach anything about GPU HA. The lesson worth keeping is the general one: a liveness signal and a serviceability guarantee are not the same thing, and on infrastructure you do not own, the gap between them can open without warning after you have already validated the system.

8. The single telemetry contract — one frame, two tiers, shared ingest

This section was held back in earlier drafts while the patent-versus-open-source question was open. That question is now settled: the project is published in full as open reference and prior art. So this section — the load-bearing design decision of the whole system — is described completely rather than redacted.

Everything else in this document is downstream of one choice: **a single versioned telemetry frame drives both tiers, through literally the same ingest code.** Not a shared *format* that each tier parses its own way — a shared *class*. The router imports it; the DNS responder imports it; they are the same forty lines making the same liveness judgment on the same bytes.

8.1 What the frame carries — and what it deliberately does not

The `TelemetryFrame` is a small, versioned, fixed-shape UDP datagram. It carries exactly three things:

- **Identity** — which node, in which pool. (`node_id`, `pool`.)
- **Sequence** — a monotonic per-node counter, so a consumer can detect reordering and loss.
- **Raw metrics** — time-to-first-token, tokens/sec, VRAM used and free, queue depth, request counts. The *numbers*, straight off the worker’s own instruments.

It carries, by strict design, **no request content** — never a prompt, never a token, never a fragment of either. And it carries **no self-asserted health** — no `status: healthy` field, no `ready: true`, nothing where the node states an opinion about its own liveness. Both omissions are load-bearing, and each pays off in a different part of the system (§8.4, §8.5).

The frame is versioned because it is a contract between components that will be deployed and upgraded independently. A worker

emitting v1 and a consumer expecting v2 is a situation the schema has to survive; the version field is how.

8.2 The shim: custom telemetry off stock software

The frame is emitted by a **telemetry shim** — a small sidecar process that scrapes a *stock, unmodified* vLLM server’s Prometheus `/metrics` endpoint, packs the numbers into the frame, and pushes it over UDP. This matters more than it sounds. It means the GPU box runs vendor software exactly as shipped — no fork, no patch, no plugin, nothing to re-port when vLLM updates. All of the project’s custom logic lives on cheap CPU boxes; the expensive GPU box stays pristine. The shim is the seam between “the vendor’s world” and “our world,” and keeping that seam thin is what makes the whole thing portable across providers and vLLM versions.

The shim also owns one non-obvious invariant: **its scrape timeout must sit well below the consumer’s freshness window**. If a scrape can block for three seconds against a three-second freshness window, a merely *slow* worker will flap in and out of eligibility — and slow is not dead. Slowness is what the TTFT score is for; death is what silence is for. Conflating them is a bug, and the fix lives in the shim’s timeout budget.

8.3 Two consumers, one class

The frame fans out over UDP to two consumers:

- The **pool’s L7 router** (local, `udp/5006`) consumes it to pick a worker per request — the fast, per-request decision.
- The **Tier-1 DNS responder** (across the internet, `udp/5106`) consumes it to decide which pools are alive enough to appear in the DNS answer — the coarse, per-pool decision.

The critical detail: these two consumers run **the same ingest class**. `TelemetryIngest` receives datagrams, validates the version, rejects stale sequence numbers, tracks per-node arrival timestamps, and answers one question — *which nodes/pools are fresh right now, by my clock?* The router asks that question with a 3-second window to choose a worker; the DNS tier asks the same question of the same code with a 10-second window to choose a pool. One implementation, two callers, two window sizes. A bug fixed in the freshness logic is fixed for both tiers at once; a property proven about the ingest holds in both tiers by construction. That is the entire payoff of “shared code, not shared format.”

8.4 Why content-free is load-bearing (not just tidy)

The frame’s refusal to carry request content is usually the first thing people would “optimize away” — surely it’d be convenient to ship a little context along with the metrics? It is exactly the wrong optimization, and the reason is architectural, not aesthetic.

Because the frame is content-free, **the telemetry path is safe to route anywhere** — including, in a productized form, to a SaaS control plane for fleet observability — *without ever touching data sovereignty*. Prompts and completions flow client-to-pool and nowhere else; the telemetry that describes the fleet’s health can flow to a dashboard, an alerting system, or an evidence report, and it is structurally incapable of leaking a prompt because it never held one. For any deployment where the whole point is that prompts cannot leave the building (regulated industries, air-gapped environments, model-IP-sensitive shops), this is the property that makes centralized observability compatible with decentralized data. Make the frame content-ful “for convenience” and you forfeit that forever. The emptiness is the feature.

8.5 Why no self-asserted health is load-bearing

The frame’s refusal to carry a health flag is the other omission, and it is the same idea as §5 stated at the level of the wire format. A node does not get to say it is healthy. It emits raw numbers and stops emitting when it dies, and **each consumer judges liveness by arrival freshness against its own clock**. There is no health endpoint to lie, no `ready` bit that stays `true` while the process wedges, no self-report to be wrong.

This is what let the system correctly handle the VRAM-zombie failure (§7.3) with no special-casing: a `kill -9`’d vLLM whose orphaned engine subprocess still pins all GPU memory *looks* alive to any check that asks the node how it’s doing — but the shim’s scrape fails, emission stops, and silence ages the node out. The frame didn’t need a “zombie detection” field because it never trusted the node’s opinion in the first place. Liveness is the consumer’s judgment, and the frame is designed to give the consumer nothing to be fooled by.

8.6 Sequence numbers, and the honesty of loss

The per-node sequence counter defends against UDP reordering — a consumer rejects a frame whose sequence is behind what it has already seen, so a delayed datagram can’t resurrect a stale reading. But it does more than that: because it is monotonic and per-node, the *gaps* in it are a measurement. When frames cross the public internet (§6), the sequence gaps are the first genuine count of internet UDP loss the project ever produced — and, tellingly, the gap count stayed at zero on the live path and only jumped at the moment a node went silent. Loss and death both show up in the sequence stream, and the frame is honest about both because it never pretends a gap didn’t happen. (The one edge this exposed — a *restarted* emitter resets its sequence to zero and is then indistinguishable from a replay, §7.1 — is the reason frame v2 adds an epoch/boot-id field: so the ingest can tell “reordered datagram” from “restarted node.”)

8.7 Why this is the contribution

Two-tier failover is not novel on its own; DNS failover is decades old, and L7 load-balancing is decades old. What is worth publishing is the **seam**: the recognition that both tiers are answering the *same liveness question at different resolutions*, and can therefore be driven by one content-free, self-report-free frame through one shared ingest — with the emptiness of the frame doing double duty as the sovereignty guarantee. That single contract is why the two tiers stay consistent, why the system is portable, why observability doesn’t compromise privacy, and why no component in the fleet can lie about being alive. It is the smallest idea in the system and the one everything else hangs from.

9. Limitations, stated plainly

Tier-1 evacuation, the whale, and the full failure arc are proven across localhost/LAN/public-internet transport; intra-pool failover is proven against real GPUs at both process level (co-resident workers) and host level (workers on separate instances), on two independent clouds (Lambda and RunPod, the latter through a NAT'd proxy). The multi-cloud orchestrator (`gpuha up/down/reap`) is built and drilled across GCP, Lambda, and RunPod. One boundary stated precisely (see §7.9): in the combined multi-cloud drill, cross-cloud evacuation and the whale were proven at the Tier-1/DNS/telemetry layer — which is the layer that actually performs evacuation — rather than with a live *Lambda* completion handoff, because Lambda's cloud blocks inbound `:9000` by default; the RunPod completion path was live, and the Lambda pool's telemetry (and thus its DNS registration and evacuation) was unaffected. That gap is a provider-default-firewall detail, not a property of the system, and closing it (a Lambda inbound rule) was deliberately not pursued. What genuinely remains is deferred hardening and breadth, not correctness gaps: a *single combined* recorded demo standing up the full multi-provider topology at once with every provider's completion path live (the pieces are each proven; the choreography and one Lambda firewall rule are the remaining step); production DNS wiring — how `api.gpuha.com` resolves on port 53, authoritative-vs-hosted-zone, anycast slaves, native CoreDNS TelemetryFrame parsing — is deferred **[D2]**; telemetry has no auth beyond source-IP scoping (frame v2: HMAC — matters more now that frames demonstrably cross the internet); control-plane HA (dual planes + a witness for split-brain) is designed but not built; cold-start optimization is characterized (cold start is pip-bound — ~130s of torch/vLLM install — not download-bound, the ~8s model pull was a red herring; the real levers are `uv` or a warm image, both product-tier); no mid-stream resume (by design — §4); scoring weights are hand-set, not learned.

10. Production path

CoreDNS hidden-master wiring (the dnshat-lineage engine) with native frame parsing; anycast slave layer; 4+ real GPUs per pool across zones; frame v2 (epoch/boot-id, HMAC, zombie signature); Envoy-grade L7 (the custom router names its own lineage: the breaker is outlier detection in miniature, the selector is a custom EDS); learned scoring weights against real TTFT distributions.

Appendix A — Environment landmines (verbatim, they cost hours)

RunPod CUDA-12.4-era driver requires pinning vLLM 0.11.0 + torch 2.8.0+cu128 + transformers<5 + hf_transfer (default pip pulls a CUDA-13 build that won't run). GCE stop/start can change internal IPs — reserve static addresses for anything referenced by config. vLLM does not auto-start on boot from a manual launch. GCP capacity reservations bill at full rate while idle. EngineCore orphan cleanup: `nvidia-smi --query-compute-apps=pid | xargs kill -9` before vLLM restart. Fake-worker port 8001 collides with RunPod pod nginx — use 8011+. Lambda has no stop state — instances bill per-minute until *terminated*, and terminate destroys the local disk (commit/push before teardown). Stopped Linodes still bill the full plan rate — delete, don't stop, for throwaway nodes. LISH/Weblish is an out-of-band serial console that always prompts for the root password and ignores SSH — it's a last-resort recovery door, not a working shell. Cloud Shell blocks outbound non-DNS UDP and disconnects on long operations — never run a workload in it; use it only as a control surface to drive a real instance. A custom-named SSH deploy key isn't offered by default — add an `~/.ssh/config` IdentityFile entry (check this first when “the key is on the repo but still permission denied”).

Current state (2026-07-03)

Running today: a permanent always-on control plane (Linode 2GB, ~\$12/mo) hosting Tier-1 (`tier1_dns.py`) and the whale (`whale.py`) as boot-enabled systemd services. Canonical code is durable in a private git repo. The two-tier telemetry contract is proven across localhost, LAN, and public-internet transport. Total project spend to date: a few dollars of GPU time plus the plane's monthly rate; five legacy boxes (~\$68/mo) were recovered and deleted, so cleanup roughly funds the plane.

Next: a *single combined* recorded demo standing up the full multi-provider topology at once (each piece is proven; the choreography is the step); breadth across more providers in one topology; **[D2]** production DNS wiring; and the deferred hardening (control-plane HA, frame v2, cold-start optimization). See TARGET_STATE.md.

[PHASE E: add demo recording links, /__stats snapshots, dig transcripts; final edit pass for tone and length.]